

PyQCU 与 QUDA MultiGrid：旧加速比撤回、double MG 修复与重新对照

PyQCU 工程分析记录

2026 年 9 月 17 日

摘要

本文撤回 2026-09-16 报告中未经对齐预算的 c64 三层 $4.3817\times$ 结论，在相同 gauge、右端项、canonical null vectors、odd-odd MATPC、停机容差、零初值和五次 median/MAD 口径下重新测量。正式 trace 环境变量全部关闭；同侧 plain BiCGStab 作为独立参考，不进入 MG 加速比。

重新构建后，QUDA 以 QUDA_PRECISION=12 同时提供 c64/c128，并启用 QUDA_MULTIGRID_DOUBLE=ON。为真正运行 double coarse MG，还需修复 double coarse-link 原子累加仍按 int 固定点解释的问题；否则 level-1 setup 会产生 NaN 或 norm 检查中止。主口径采用 --quda-strategy aligned，即双侧都是一次递归 V-cycle；--quda-strategylibrary 保留 QUDA 原生 CA-GCR、nu_post=8 与 coarse maxiter 16，作为敏感性对照。

当前可信的大格 c64 三层主结果为 PyQCU 0.654446 s、QUDA 1.299921 s，加速比 $1.9863\times$ ，外迭代数 14/39，真残差分别为 4.90135×10^{-7} 与 7.32826×10^{-7} 。c128 三层在 $16^3 \cdot 16$ 上得到 PyQCU 0.365517 s、QUDA 1.290496 s，加速比 $3.5306\times$ ，外迭代数 44/87。c128 三层 $16 \cdot 32 \cdot 32 \cdot 48$ 的全 double setup 在 32 GiB V100 上双侧 OOM，因此不报告跨库大格 c128 加速比。

目录

1	旧结果撤回与验收口径	2
2	实现与修复	2
2.1	默认关闭的 trace 与正式闸门	2
2.2	BiCGStab 参考与 QUDA 指针修复	3
2.3	QUDA double MultiGrid	3
3	正式协议	4
3.1	输入与计时	4
3.2	aligned 与 library 两种 QUDA 策略	4
4	主结果	4
5	BiCGStab 参考与失真检查	5

1 旧结果撤回与验收口径	2
6 trace 开销	5
7 正确性与残差	6
8 显存与多卡边界	6
8.1 c128 大格点	6
8.2 QUDA mixed-coarse 容量补充	6
8.3 多卡	6
9 复现命令与证据	7
10 结论	8

1 旧结果撤回与验收口径

旧 c64 16·32·32·48 三层结果 0.653995 s 对 2.865627 s、 $4.3817\times$ 不再作为正式结论。重新检查后确认旧 QUDA 记录使用了与 PyQCU 不同的 smoother 与中间层 coarse solver 预算：PyQCU 每个外层迭代执行一次递归 V-cycle，而旧 QUDA 配置在中间层执行多轮内层迭代。新旧差异不是浮点残差门限或计时边界造成的。

本轮验收标准为：

1. 新增性能测试功能默认关闭；正式速度比必须在 trace 环境变量为空时产生；
2. PyQCU 与 QUDA 使用同一 gauge、RHS、canonical full null vectors、mass、block、coarse spin、odd-odd MATPC 和相对真残差门限；
3. 至少覆盖二、三层，至少覆盖 c64/c128，并扩展到三个格点体积；
4. 每侧记录 MG steady solve、独立真残差，以及同侧 plain BiCGStab 参考时间与真残差；
5. 对小格点另测 trace-on，明确量化 trace 对性能的影响；
6. 对 c128 double MG 做真实 setup/solve 验证，不能只以编译成功作证据；
7. 在硬件能力范围内给出可追溯的正式 JSON，无法运行的条件明确标记。

2 实现与修复

2.1 默认关闭的 trace 与正式闸门

PyQCU 使用 PYQCU_STRICT_TRACE_FILE，QUDA 使用 QUDA_MG_TRACE_FILE。两者未设置时不创建 trace 文件，也不分配 trace 专用 workspace。正式 profile 若检测到任一变量存在，会在启动 worker 前失败；诊断运行必须显式传 `--allow-trace`。这样可避免把分层残差计算、同步和字符串 I/O 混入正式 speedup。

2.2 BiCGStab 参考与 QUDA 指针修复

PyQCU 参考调用 `applyCloverBistabCgQcu`; QUDA 参考将 `inv\_type` 设为 `QUDA\_BICGSTAB\_INVERTER`。仅把 `inv\_type\_precondition` 设为 `INVALID` 不够, 因为 PyQUDA 的 `getClover()` 仍保留 MG 实例指针。QUDA 的 solver factory 会把 BiCGStab 判定为“不支持显式预条件器”并终止。修复后同时设置 `inv\_type\_precondition=QUDA\_INVALID\_INVERTER` 与 `invert.preconditioner=Pointer("void")`。

参考解与 MG 使用相同 tolerance、maxiter、零初值、误差复算和真残差门限, 但被明确标记为 `excluded_from_speedup=true`, 不参与 `speedup_pyqcu_over_quda`。

2.3 QUDA double MultiGrid

上游 QUDA 1.1.0 默认关闭 double recursive MultiGrid。为得到可运行的 c128 coarse 路径, 本轮做了以下最小修复:

1. CMake 增加 `QUDA_MULTIGRID_DOUBLE`, 开启时定义 `GPU\_MULTIGRID\_DOUBLE`;
2. `matrix\_tile.cuh` 对 mixed precision gauge accessor 显式构造目标 `complex<T>`;
3. double coarse-link 的 shared/global atomic accumulation 使用 `double` 存储, 而不是把 `double` 字段按 `int` 固定点解释;
4. c64 仍保留确定性 `int` fixed-point 路径, 避免回归性能与行为。

关键实现位于:

- `refer/git-rep/quda/include/kernels/coarse_op_kernel.cuh`
- `refer/git-rep/quda/lib/coarse_op.in.cu`
- `refer/git-rep/quda/lib/coarsecoarse_op.hpp`
- `refer/git-rep/quda/lib/staggered_coarse_op.in.cu`

统一构建使用:

```
QUDA_PRECISION=12
QUDA_RECONSTRUCT=7
QUDA_INTERFACE_QDP=ON
QUDA_QIO=ON / QUDA_QMP=ON
QUDA_MULTIGRID_NVEC_LIST=12,24
QUDA_MULTIGRID_DOUBLE=ON
QUDA_ENABLE_MMA=OFF
QUDA_GPU_ARCH=sm_70
```

3 正式协议

3.1 输入与计时

两侧共享以下输入与边界：

- fine Wilson/Clover、mass 0.05、canonical 12 个 full near-null vectors；
- odd-odd MATPC, `coarse\_spin=2`, block [2, 2, 2, 2]；
- c64 tolerance 10^{-6} 、真残差门限 5×10^{-6} ；
- c128 tolerance 10^{-8} 、真残差门限 5×10^{-8} ；
- 零初值、2 次 warmup、5 次 measured solve、median/MAD；
- steady solve 由调用方自有输入输出计时，setup、输入 I/O、真残差复算分开；
- 正式运行设置 QUDA_RESOURCE_PATH 复用调优参数。

3.2 aligned 与 library 两种 QUDA 策略

`aligned` 将两侧统一到“一个外层迭代执行一次递归 V-cycle”：MR smoother、`smoother\_tol=0`、非最粗层 `coarse\_solver\_maxiter=1`，最粗层仍使用 200。

`library` 保留 QUDA/PyQUDA 默认：CA-GCR smoother、`smoother\_tol=0.25`、`nu\_pre=0`、`nu\_post=8`、`coarse\_solver\_maxiter=16`。该策略只用于判断结论是否依赖某套预算，不与 `aligned` 数字混合计算。

4 主结果

表 1: aligned 主口径的五次 median；所有正式运行均 trace-off。

格点 / 层 / 精度	PyQCU (s)	QUDA (s)	加速比	外迭代 Py/QUDA
$8^3 \cdot 16 / 2 / \text{c64}$	0.054184	0.399675	7.3763	9 / 32
$8^3 \cdot 16 / 3 / \text{c64}$	0.043200	0.516422	11.9542	10 / 33
$16^3 \cdot 16 / 3 / \text{c64}$	0.149240	0.745390	4.9946	31 / 59
$16 \cdot 32 \cdot 32 \cdot 48 / 2 / \text{c64}$	1.944663	2.183002	1.1226	11 / 37
$16 \cdot 32 \cdot 32 \cdot 48 / 3 / \text{c64}$	0.654446	1.299921	1.9863	14 / 39
$8^3 \cdot 16 / 3 / \text{c128}$	0.077162	0.975873	12.6471	14 / 46
$16^3 \cdot 16 / 3 / \text{c128}$	0.365517	1.290496	3.5306	44 / 87

小格点加速比明显大于中型/大型格点，原因是固定 setup 之外的外层迭代数差异在小体积上占主导；大格点则同时受每层算符规模、缓存与固定 kernel 开销影响。不能仅凭单一小格点外推正式部署性能。

同一 $16 \cdot 32 \cdot 32 \cdot 48$ c64 格点上, 二层为 $1.1226\times$, 三层为 $1.9863\times$ 。这不是单点偶然值, 而是层数消融: 增加第三层后 PyQCU 时间从 1.944663 s 降至 0.654446 s, QUDA 从 2.183002 s 降至 1.299921 s, 两侧均从更深粗化中获益, 但 PyQCU 的降幅更大。

表 2: QUDA 原生 library 策略敏感性; 不替代表 1 主口径。

格点 / 层 / 精度	PyQCU (s)	QUDA library (s)	加速比
$16^3 \cdot 16$ / 3 / c64	0.170131	0.465514	2.7362
$16 \cdot 32 \cdot 32 \cdot 48$ / 3 / c64	0.654533	2.462287	3.7619
$16^3 \cdot 16$ / 3 / c128	0.357061	0.961293	2.6922

5 BiCGStab 参考与失真检查

同侧 BiCGStab 不进入 MG-vs-MG 加速比, 但用于识别计时或配置失真。

表 3: 大格 c64 三层: MG 与同侧 plain BiCGStab median。

实现	MG (s)	BiCGStab (s)
PyQCU	0.654446	1.709560
QUDA	1.299921	0.329646

表中 QUDA 的 MG 慢于其 plain BiCGStab, 说明 aligned 的“一次递归”配置虽然数学上对齐, 却不是 QUDA 在该格点上的绝对最快配置; library 敏感性表则给出了另一侧证据。因此本文同时报告两套策略, 但主结论仍使用 aligned, 因为它排除了“QUDA 多执行内层粗迭代而 PyQCU 只执行一次递归”的口径差。

6 trace 开销

小格 c64 三层同一参数下, trace-off 取正式文档的 median, trace-on 取诊断文档的 steady median。trace-on 数据只用于量化观测代价, 不参与加速比。

表 4: trace-off 与 trace-on 的 steady median。

模式	PyQCU (s)	QUDA (s)	说明
trace-off	0.043971	0.505706	正式主口径
trace-on	0.125649	0.560432	诊断, excluded
开销因子	$2.86\times$	$1.11\times$	比较同侧

PyQCU 的 trace 需要在 C++ 内记录逐层残差与阶段, 并执行额外同步, 因此对微小格点的相对开销远高于 QUDA 的原生日志; 这也验证了正式速度比必须在 trace-off 下产生。

7 正确性与残差

所有正式点均通过独立全 Wilson/Clover 真残差复算。大格 c64 三层的最大相对真残差为 PyQCU 4.90135×10^{-7} 、QUDA 7.32826×10^{-7} ；中型 c128 三层分别为 1.83242×10^{-8} 与 1.83035×10^{-8} 。两库输出均来自同一 RHS，且解在进入残差复算前按约定完成归一化转换。

8 显存与多卡边界

8.1 c128 大格点

c128 三层 $16 \cdot 32 \cdot 32 \cdot 48$ 的全 double hierarchy setup 在 32 GiB V100 上双侧 OOM。PyQCU 最初在 block orthogonalization 请求额外 864 MiB 时失败；将 CGS 从三矩阵副本降为两缓冲后，该点可通过，但随后在 `to\_qcu\_blocked()` 导出 blocked basis 时仍 OOM。进一步把 blocked 布局改为正交化时直接分配的主存储后，导出已成为零拷贝；setup 继续推进到 Galerkin 粗块分配，最终仍因整套 double hierarchy 超出 32 GiB 而 OOM。这说明限制不是单个临时数组，而是 fine fields、blocked basis、粗块与 coarse assets 的联合驻留。QUDA 则在 double coarse-link setup 请求 75 MiB 时失败。把 PyQCU 层数从三层降到二层后仍在同一个 `build\_strict\_galerkin\_colored()` 粗块分配点 OOM，进一步说明大格点不可达的主导因素是全 double 的 fine/coarse 精度预算，而不是额外第三层本身的体积。

8.2 QUDA mixed-coarse 容量补充

为区分“QUDA 算法不可用”与“全 double 容量不足”，另运行了 `--quda-coarse-precisionsingle` 的 QUDA-only 点：外层仍为 c128，QUDA coarse MG 使用 single precision，gate 仍由独立 c128 真残差复算。 $16 \cdot 32 \cdot 32 \cdot 48$ 三层成功，setup 约 617 s，60 次外层迭代的 steady solve 为 2.265 s，最终相对真残差 1.79×10^{-8} ；运行期间设备峰值约 9.8 GiB。

该点证明了 QUDA 的 c128 mixed-coarse 路径可用并显著降低显存占用，但 PyQCU strict 当前显式要求各层相同 dtype，尚未实现同等 mixed-coarse runtime，因此不能把 QUDA-only 数字写入跨库 speedup。PyQCU 的 block-precision 低内存选项虽把 coarse blocks 降为单精度，仍在 batch diagonal inverse 阶段 OOM；这作为后续 strict mixed-precision 实现的入口，而不是已完成结论。失败文档保留在 `data/mg_matrix_20260916_round2/formal-c128-13/benchmark.json`。因此 c128 最大可信跨库正式点是 $16^3 \cdot 16$ 三层；小格 c128 三层用于确认 double coarse MG 的真实 setup/solve，而不是用单侧结果伪造比值。

8.3 多卡

当前运行环境只有一张 V100 属于 PyTorch 构建支持的 sm_70+ 架构。两张 P100 为 sm_60，当前 PyTorch 支持列表不含 sm_60，QUDA 本轮也按 sm_70 构建。因此不能把 P100 并发纳入本轮正式 MultiGrid 结论；这属于硬件/工具链能力缺口，不是算法验证失败。阶段 1 MPI 能力门禁仍在可用范围内通过：单 rank preflight 为 18 passed/9 skipped，`mpirun-np2` 每个 rank 为 14 passed/13 skipped；这些测试只确认 c64/c128 全局 dot/norm 归约与 fail-closed preflight，不能解释为已实现分布式 halo 或 distributed fused FGMRES。

9 复现命令与证据

```

source ./env.sh
source examples/qcu/dev87/quda_env.sh
export QUDA_RESOURCE_PATH=/root/PyQCU/data/quda-resource-cache
export QUDA_ENABLE_TUNING=1
unset PYQCU_STRICT_TRACE_FILE QUDA_MG_TRACE_FILE

# 主口径：正式大格 c64 三层
python -B examples/qcu/dev87/bench_strict_vs_quda.py \
  --profile formal --side both --quda-strategy aligned \
  --lattice 16 32 32 48 --levels 3 --precision c64 \
  --repeats 5 --cache-expect hit \
  --reference-solver bicgstab --reference-warmups 1 \
  --gauge-path data/gauge_16x32x32x48_m0.05_seed42_c64.h5 \
  --nullvec-path data/L16x32x32x48_nvec12_full_c64.h5 \
  --quda-nullvec-prefix data/L16x32x32x48_nvec12_quda \
  --quda-nullvec-manifest data/L16x32x32x48_nvec12_quda.conversion.json \
  --output data/mg_matrix_20260916_round2/formal-c64-l3/benchmark.json

# trace 诊断：正式加速比之外单独报告
python -B examples/qcu/dev87/trace_strict_vs_quda.py \
  --profile smoke --repeats 2 --lattice 8 8 8 16 --levels 3 \
  --precision c64 --cache-expect any \
  --reference data/mg_matrix_20260916_round2/trace-reference-small-c64-l3.json \
  --output data/mg_matrix_20260916_round2/trace-small-c64-l3.json

```

表 5: 主要证据文件

文件	内容
data/mg_matrix_20260916_round2/formal-c64-l3/benchmark.json	当前 c64 大格三层 aligned 五次正式记录。
data/mg_matrix_20260916_round2/formal-c64-l3-medium/benchmark.json	c64 中格三层 formal。
data/mg_matrix_20260916_round2/formal-c64-l2-small/benchmark.json	c64 小格二层 formal。

data/mg_matrix_20260916_round2/formal-c64-l3-small/benchmark.json	c64 小格三层 formal。
data/mg_matrix_20260916_round2/formal-c128-l3-small/benchmark.json	c128 小格三层 formal, double MG 路径。
data/mg_matrix_20260916_round2/formal-c128-l3-medium/benchmark.json	c128 中格三层 formal, double MG 路径。
data/mg_matrix_20260916_round2/library-formal-c64-l3/benchmark.json	QUDA library 策略敏感性。
data/mg_matrix_20260916_round2/trace-small-c64-l3.json	trace-on/off 小格诊断与残差阶段。

10 结论

本轮得到三点可复核结论：

1. 旧 c64 $4.3817\times$ 来自未对齐内层预算，已撤回；当前大格 c64 三层 aligned 主结果为 $1.9863\times$ ，五次样本和真残差均通过。
2. QUDA double MG 已从“不可用”推进到 c128 小/中格点真实可运行，但全 double 大格三层受 32 GiB V100 显存限制，不能伪造跨库比值。
3. trace 功能对 PyQCU 小格点有显著观测开销，正式加速比必须在 trace-off 下采集；BiCGStab 参考和 aligned/library 双策略已作为后续防失真硬闸门写入技能库。